

Dictionaries

A dictionary is a built-in data type that represents a collection of **key-value pairs**.
Dictionaries are enclosed in **curly braces {}**.

Example:

```
empty_dict = dict() # Creating an Empty  
S_dict = {"Name": "Chandhash", "Age": 19}
```



The diagram illustrates the components of a dictionary entry. A purple arrow points from the word 'key' to the string 'Name' in the dictionary literal. An orange arrow points from the word 'value' to the string 'Chandhash' in the same entry.

Accessing Values

You can access the **values** in a dictionary using their corresponding **keys**.

Syntax:

```
Value = dict_name["key_name"]
```

Example:

```
Person = {"Name": "Ram", "Age": 18}
name = Person["Name"]
print(name) # Ram
print(Person["Age"]) # 18
```

Add or modify

Inserts a new key-value pair into the dictionary. If the key already exists, the value will be updated; otherwise, a new entry is created.

Syntax:

```
dict_name[key] = value
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
Person["Country"] = "India" # A new entry will be created.  
Person["City"] = "Delhi" # Update the existing value for the same key  
print(Person) # {'Name': 'Ram', 'Age': 19, 'City': 'Delhi', 'Country': 'India'}
```

del

Removes the specified key-value pair from the dictionary. Raises a **KeyError** if the key does **not exist**.

Syntax:

```
del dict_name[key]
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Delhi"}  
del Person["Age"]  
print(Person) # {'Name': 'Ram', 'City': 'Delhi'}
```

update()

The `update()` method merges the provided dictionary into the existing dictionary, adding or updating key-value pairs.

Syntax:

```
dict_name.update({key: value})
```

Example:

```
person = {"Name": "Ram", "City": "Delhi"}  
person.update({"Branch": "B.Tech AI & ML"})  
print(person) # {'Name': 'Ram', 'City': 'Delhi', 'Branch': 'B.Tech AI & ML'}
```

clear()

The `clear()` method empties the dictionary, **removing all key-value pairs within it**. After this operation, the dictionary is still accessible and can be used further.

Syntax:

```
dict_name.clear()
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
Person.clear()  
print(Person) # {}
```

key existence

You can check for the existence of a key in a dictionary using the in keyword

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
If "Name" in Person:  
    print("Name exists in the dictionary.")
```

copy()

Creates a **shallow copy** of the dictionary. The **new dictionary contains the same key-value pairs as the original**, but they remain distinct objects in memory

Syntax:

```
new_dict = dict_name.copy()
```

Example:

```
Person = {"Name": "Ram", "Age": 19}
New_person = Person.copy()
print(New_person) # {'Name': 'Ram', 'Age': 19}
```

keys()

Retrieves **all keys** from the dictionary and **converts them into a list**. Useful for iterating or processing keys using list methods.

Syntax:

```
keys_list = list(dict_name.keys())
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
all_keys = list(Person.keys())  
print(all_keys) # ['Name', 'Age', 'City']
```

values()

Extracts **all values** from the dictionary and **converts them into a list**. This list can be used for further processing or analysis.

Syntax:

```
values_list = list(dict_name.values())
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
all_values = list(Person.values())  
print(all_values) # ['Ram', 19, 'Bhagalpur']
```

items()

Retrieves all key-value pairs as tuples and converts them into a list of tuples. Each tuple consists of a key and its corresponding value.

Syntax:

```
items_list = list(dict_name.items())
```

Example:

```
Person = {"Name": "Ram", "Age": 19, "City": "Bhagalpur"}  
info_l = list(Person.items())  
print(info_l) # [('Name', 'Ram'), ('Age', 19), ('City', 'Bhagalpur')]
```

Set

A set is an **unordered** collection of **unique elements**.

Syntax:

```
set_name = {elements}
```

Example:

```
empty_set = set() #Creating an Empty  
set_a = {5,6,9,"Kali"}  
print(set_a) # {9, 'Kali', 5, 6}
```

add()

Elements can be added to a set using the `add()` method. Duplicates are **automatically removed**, as sets only store **unique values**.

Syntax:

```
set_name.add(element)
```

Example:

```
set_a = {5,6,9,"Kali"}  
set_a.add("Apple")  
print(set_a) # {5, 6, 'Apple', 9, 'Kali'}
```

clear()

The `clear()` method removes all elements from the set, resulting in an empty set. It updates the set in-place.

Syntax:

```
set_name.clear()
```

Example:

```
set_a = {5,6,9,"Kali"}  
set_a.clear()  
print(set_a) # set()
```

copy()

The `copy()` method creates a shallow copy of the set. **Any modifications** to the copy **won't affect the original set**.

Syntax:

```
new_set = set_name.copy()
```

Example:

```
set_a = {5,6,9,"Kali"}  
set_b = set_a.copy()  
print(set_b) # {5, 6, 9, 'Kali'}  
set_b.clear()  
print(set_a) # {5, 6, 9, 'Kali'}  
print(set_b) # set()
```

discard()

Use the `discard()` method to **remove a specific** element from the set. **Ignores** if the element is **not found**.

Syntax:

```
set_name.discard(element)
```

Example:

```
set_a = {5,6,9,"Kali"}  
set_a.discard("Kali")  
print(set_a) # {5,6,9}
```

issubset()

The `issubset()` method checks if the current set is a subset of another set. It returns True if all elements of the current set are present in the other set, otherwise False.

Syntax:

```
is_subset = set1.issubset(set2)
```

Example:

```
set1 = {5,3,6,8,4}  
set2 = {5,3,6,8,4}  
is_subset = set1.issubset(set2)  
print(is_subset) # True
```

issuperset()

The `issuperset()` method checks if the current set is a superset of another set. It returns True if all elements of the other set are present in the current set, otherwise False.

Syntax:

```
is_superset = set1.issuperset(set2)
```

Example:

```
set1 = {5,3,6,8,4}
```

```
set2 = {5,3,6,8,7,4}
```

```
is_superset = set1.issuperset(set2)
```

```
print(is_subset) # True
```

pop()

The `pop()` method removes and **returns an arbitrary element** from the set. It raises a `KeyError` if the set is **empty**. Use this method to remove elements when the **order doesn't matter**.

Syntax:

```
removed_element = set_name.pop()
```

Example:

```
set1 = {5,3,6,8,4}  
removed_element = set1.pop()  
print(removed_element) # 4
```

remove()

Use the `remove()` method to remove a specific element from the set. Raises a `KeyError` if the element is not found.

Syntax:

```
set_name.remove(element)
```

Example:

```
set1 = {5,3,6,8,4}  
set1.remove(6)  
print(set1) # {3, 4, 5, 8}
```

Set Operations

Perform various operations on sets: `union`, `intersection`, `difference`, `symmetric\_difference`.

Syntax:

```
union_set = set1.union(set2)
intersection_set = set1.intersection(set2)
difference_set = set1.difference(set2)
sym_diff_set = set1.symmetric_difference(set2)
```

Example:

```
set1 = {5,3,6,8,4}
set2 = {5,3,6,8,7,4}
union_set = set1.union(set2)
intersection_set = set1.intersection(set2)
difference_set = set1.difference(set2)
sym_diff_set = set1.symmetric_difference(set2)
print(union_set) # {3, 4, 5, 6, 7, 8}
print(intersection_set) # {3, 4, 5, 6, 8}
print(difference_set) # set()
print(sym_diff_set) # {7}
```

update()

The `update()` method adds elements from another iterable into the set. It maintains the **uniqueness** of elements.

Syntax:

```
set_name.update(iterable)
```

Example:

```
set1 = {5,3,6,8,4}  
set1.update([6,9])  
print(set1) # {3, 4, 5, 6, 8, 9}
```



Thank you!

Author:

Chandrasah Kumar